

Intelligent Robotics Assignment 2

Michele Zarantonello 2162530

Riccardo Grosso 2155345

Jan Luca Michael Cordes 2158346

{michele.zarantonello, riccardo.grosso.1, janlucamichael.cordes}@studenti.unipd.it

Github Repo Video

Github accounts: michelezarantonello, groricc, lucordes

January 18, 2026

1 Introduction

Basic goal: The goal is to swap two cubes (parallelepipeds but here referred as cubes) with a robot (UR5). The cubes are red and blue and have apriltags on their top. Pipeline idea:

- detection of the Apriltag
- move the arm to the Apriltag
- safe swap of the cubes (this means we don't want to have collisions)
- return to the home position

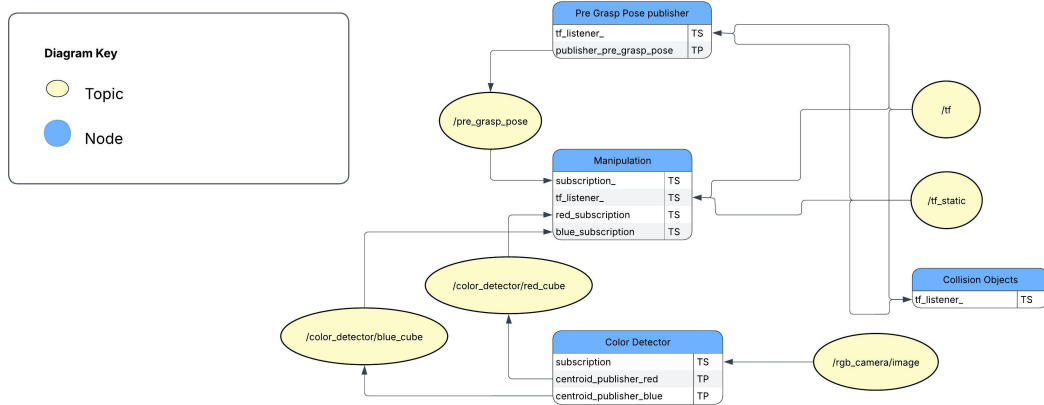


Figure 1: Overall structure

In Fig. 1 the nodes developed and all the topics related are shown. TP (Topic Publisher) and TS (Topic Subscriber) are related to the attributes of the Node. If the node listens on that topic it is referred as TS, otherwise when it is publishing on that topic it is referred as TP.

2 Definition of the collision objects

This node periodically queries the TF tree. When the AprilTags corresponding to the red and blue cubes are detected, box-shaped collision objects are added to the MoveIt planning scene.

During motion planning the MoveIt planner takes these collision objects and any defined constraints into account to generate collision-free trajectories between poses.

For each cube the following steps are performed:

- The transform between the AprilTag frame and the `base_link` frame is looked up using TF.
- The retrieved transform is converted into a pose.
- Small manual position offsets are applied to better align the collision object with the physical cube.
- A box-shaped collision object named `cube1/cube10` is added to the MoveIt planning scene.

3 Developed Nodes

3.1 pregraspposenode

The node converts a pre-defined pose relative to an AprilTag (20cm above it) into the robot's base frame and publishes it as a pre-grasp target, so the robot can use it to plan a motion toward grasping the object.

The function `send_goal` publish the pose (pre grasp pose) on the `/pre_grasp_pose` topic.

3.2 Manipulation

Manipulation node provides all the movements done by our manipulator. This node subscribes to `/pre_grasp_pose` topic for the initial movement right above the red cube. No publisher/subscriber convention is used for the rest of the node. The basic pipeline of this node is the following: pre-grasp - open gripper - cartesian descend - close gripper - cartesian ascend - pre drop - drop (open gripper). In the end we return to home state. Some joints constraints are applied to shoulder pan and shoulder lift joints, and then push them back into the constraints vector.

See Fig 2 to the the sequence of how/what functions are getting called can be seen.

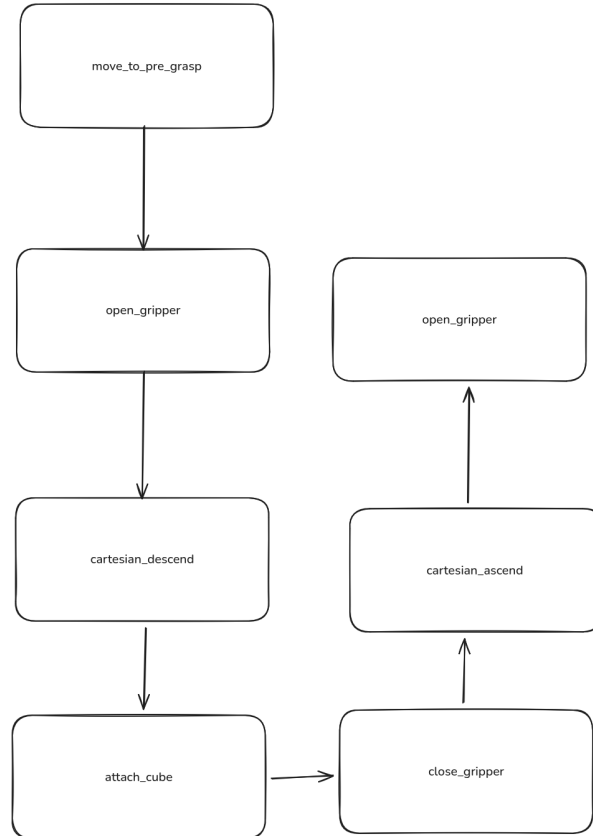


Figure 2: Flowchart of the manipulation order

Challenges And How We Solved Them

The first problem we encountered was related to the cartesian descend path. In particular, even if the gripper was positioned 20 cm perfectly above the cube, occasionally the cartesian descend was failing. This was due to the fact that a **robot link was colliding with the table** during the descend (these collisions were occasional because they depended on the arm's specific kinematic configuration prior to the descent, which varied with each attempt). We tackled this problem by **applying constraints on two joints**, thus preventing the collision during the descend.

Another challenge that has required significant time was the **proper calibration of the parameters**. In particular

- find the right offset for moving to every pre-grasp pose (along each axis)
- understand how many cm should the gripper descend to grab the cubes
- identify the proper values for the joint to be able to securely grab the cubes without causing them to fall

Regarding this final point, initially we attempted to use the "close" robot state but, also by the help of RViz visual interface, we were able to understand that the "SUCCESS" error code was triggered only when the gripper was completely closed (reached the desired joint value). This was never happening (of course) due to the presence of the cube inside the gripper.

One last problem or concern that we had was that the manipulator behaviour was not robust as desired, meaning for example that during some tests the **cube was falling** or the gripper was not closing correctly. We tackle this problem by adding some constraint, thus making the robotic arm movements more predictable.

Finally, we faced an issue when attempting to load the MoveIt configurations using MoveItConfigs-Builder in the launch file, as an "invalid path" error persisted. By examining the `ur_gripper.urdf.xacro` file located in `ir_2526/ir_movit_config/config`, we identified a directory mismatch: the file attempted to include a Xacro file from a `/urdf/` folder that did not exist, as the files were actually stored in the `/config/` folder. Although we initially intended not to modify the `ir_2526` directory, we corrected the file by commenting out the invalid inclusion and adding the following lines:

```
<xacro:include filename="$(find ir_movit_config)/config/robot_with_environment.urdf.xacro" />

<xacro:include filename="$(find ir_movit_config)/config/ur_gripper.ros2_control.xacro" />
```

By correcting the path resolution in this manner, we were able to resolve the configuration error.

4 Extra Part: Color Detection

The color detection process is done through the *ColorDetector* node, which is responsible for detecting two colored objects (a red cube and a blue cube) in an RGB camera stream. It performs color-based image segmentation using OpenCV library, computes the 2D image-plane centroids of the detected objects, and publishes these centroids as stamped geometric points. The node is designed to use a color camera stream where cubes with known and distinguishable hues are present. Pipeline of the node functioning:

- Subscription to an RGB camera image topic
- Conversion of ROS images to OpenCV format
- Conversion from BGR to HSV color space
- Color detection in HSV space
- Computation of centroids of detected colored regions
- Publication of centroid positions at a controlled rate

RESULTS

We successfully completed the swap task and the cube color detection, though some inconsistencies remain. Specifically, during the swap phase, the gripper is not always capable of picking the blue cube; even when an initial grasp is successful, the cube occasionally falls during transport. We attempted to manually tune the gripper joint values for the blue cube, as we did for the red one, but were unable to achieve total reliability. Currently, the success rate for the blue cube remains approximately one out of every two or three attempts.